



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**DESIGN AND DEVELOPMENT OF AN INVENTORY  
MANAGEMENT MODULE FOR AN INTELLIGENT  
SUPERMARKET MANAGEMENT SYSTEM**

**A CAPSTONE PROJECT REPORT**

*A Capstone Project Report submitted in partial fulfilment of the requirements  
for the Course of*

**CSA-0908 – Programming in Java**

*to the award of the degree of*

**BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**

**Submitted by**

**B. DEEPTHI                      192424269**

**Under the Supervision of**

**Dr. E. MEGANATHAN**

**SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

**SEPTEMBER 2026**





# **SIMATS ENGINEERING**



Saveetha Institute of Medical and Technical Sciences

Chennai-602105

## **BONAFIDE CERTIFICATE**

This is to certify that the Capstone Project entitled “Design and Development of an Inventory Management Module for an Intelligent Supermarket Management System” has been carried out by B. Deepthi (192424269) under the supervision of Dr. E. Meganathan and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

### **COURSE COORDINATOR DETAILS :**

Dr.G.Charlyn Pushpa Latha,  
Professor/Dept of CSE,  
Saveetha School of Engineering, SIMATS,  
Chennai,

### **COURSE FACULTY**

Dr. E. Meganathan,  
Designation  
Department of Computer Science and Engineering  
SIMATS Engineering,  
SIMATS, Chennai – 602105.



# SIMATS ENGINEERING



Saveetha Institute of Medical and Technical Sciences

Chennai-602105

## DECLARATION

We, M. Hashetha, Chethrika Sri K M and B. Deepthi of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Design and Development of an Inventory Management Module for an Intelligent Supermarket Management System” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

**Signature of the Students with Names**

B.Deepthi

192424269



# SIMATS ENGINEERING



Saveetha Institute of Medical and Technical Sciences

Chennai-602105

## Individual Contribution Statement

*(Mandatory for all team projects)*

| Student Name / Register No.    | Specific Responsibilities                                                                                 | Design & Development Contribution                                                                  | Testing & Analysis Contribution                                              | Report Contribution       | Approx. Contribution (%) |
|--------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------|--------------------------|
| M. Hashetha<br>192521162       | Design of the Inventory Management module: Product data model, database schema and low-stock alert logic. | Designed the Product entity, MySQL schema, and DAO layer for the inventory module.                 | Unit-tested product CRUD operations and validated stock-quantity edge cases. | Chapters 1, 3             | 35%                      |
| Chethrika Sri K M<br>192511112 | Development of the inventory web interface and servlet-based business logic.                              | Implemented Add/Update/Delete product screens using JSP, Servlets and Bootstrap.                   | Functional testing of the stock management UI and low-stock alert display.   | Chapters 4                | 35%                      |
| B. Deepthi<br>192424269        | Integration, JDBC connectivity and documentation of the inventory module.                                 | Integrated the inventory module with the MySQL database via JDBC and deployed it on Apache Tomcat. | System-level testing, results analysis and comparison with existing systems. | Chapters 2, 5, References | 30%                      |
| Total                          |                                                                                                           |                                                                                                    |                                                                              |                           | 100%                     |

## ABSTRACT

Supermarkets stock hundreds of fast-moving products, and the accuracy of business decisions such as reordering, pricing and shelf allocation depends entirely on how reliably that stock is recorded. When product master data and stock quantities are tracked manually on registers or spreadsheets, entries are frequently delayed, duplicated or miskeyed, which leads to stock-outs, overstocking, and an inability to answer a simple question such as “what do we have right now?” This engineering problem – the lack of a single, accurate, real-time record of inventory – forms the focus of this capstone project.

The project designs and develops the Inventory Management module of a larger Intelligent Supermarket Inventory, Expiry, and Sales Management System. The module allows staff to add, update, categorise and delete product records; continuously track available stock quantities against a configurable reorder level; and automatically raise low-stock alerts so that replenishment decisions can be made before a product runs out. The system is significant because it removes the manual bookkeeping step that is the single largest source of stock-record error in a small-to-medium supermarket, while remaining light enough to run on commodity hardware.

The module was implemented as a Java web application using JSP and Servlets for the presentation and business-logic tiers, a MySQL relational database for persistent storage, and JDBC as the connectivity layer between them. The front end was built with HTML, CSS, JavaScript and Bootstrap for a responsive interface, developed in Eclipse IDE and deployed on Apache Tomcat, with Git/GitHub used for version control. The development followed an iterative methodology: requirement analysis, entity-relationship and schema design for the Product entity, implementation of the Data Access Object (DAO) layer, construction of the stock-management screens, and finally unit, functional and system-level testing against a defined acceptance criterion for each requirement.

The resulting module was verified against a live product catalogue (six representative SKUs spanning packaged, liquid and perishable goods) and correctly performed create, read, update and delete operations, quantity tracking by unit, and low-stock flagging, with all defined test cases passing. The module forms a stable, extensible foundation for the Expiry Management and Sales & Analytics modules that complete the parent system, and demonstrates that a disciplined.

## TABLE OF CONTENTS

|     |                                                                                                   |    |
|-----|---------------------------------------------------------------------------------------------------|----|
| i   | CERTIFICATE FROM THE SUPERVISOR.....                                                              | 3  |
| ii  | DECLARATION BY THE CANDIDATE.....                                                                 | 4  |
| iii | INDIVIDUAL CONTRIBUTION STATEMENT .....                                                           | 5  |
| iv  | ABSTRACT .....                                                                                    | 6  |
| v   | LIST OF FIGURES.....                                                                              | 8  |
| vi  | LIST OF TABLES .....                                                                              | 9  |
| vii | LIST OF ABBREVIATIONS .....                                                                       | 10 |
| 1   | CHAPTER 1: Introduction and Engineering Problem.....                                              | 11 |
| 2   | CHAPTER 2: Literature Review and Existing Solutions .....                                         | 14 |
| 3   | CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security ..... | 16 |
| 4   | CHAPTER 4: Implementation, Testing & Results, Project Management .....                            | 21 |
| 5   | CHAPTER 5: Conclusion & Reflection.....                                                           | 26 |
|     | References .....                                                                                  | 28 |
|     | Appendices .....                                                                                  | 29 |

## LIST OF FIGURES

| Figure No. | Figure Name                                                                                         | Page No. |
|------------|-----------------------------------------------------------------------------------------------------|----------|
| 3.1        | System Architecture of the Intelligent Supermarket Management System (Inventory Management context) | 18       |
| 4.1        | Product.java – Product Entity Class (Model Layer)                                                   | 21       |
| 4.2        | Inventory Module – Stock Management Screen (Product List View)                                      | 22       |

## LIST OF TABLES

| Table No. | Table Name                                                     | Page No. |
|-----------|----------------------------------------------------------------|----------|
| 2.1       | Comparative Analysis of Existing Inventory Solutions           | 14       |
| 3.1       | Stakeholder / User Requirements                                | 16       |
| 3.2       | Functional & Non-Functional Requirements                       | 16       |
| 3.3       | Constraints & Applicable Standards                             | 17       |
| 3.4       | Design Specifications                                          | 17       |
| 3.5       | Alternative Solutions Evaluation Matrix                        | 18       |
| 3.6       | Trade-off Analysis                                             | 19       |
| 3.7       | Sustainability, Ethics, Safety, Risk & Security                | 19       |
| 3.8       | Risk Register                                                  | 20       |
| 4.1       | Tools, Software and Equipment Used                             | 22       |
| 4.2       | Test Plan & Verification                                       | 22       |
| 4.3       | Results – Achieved vs. Required Values                         | 23       |
| 4.4       | Comparison with Existing Solutions & Achievement of Objectives | 24       |
| 4.5       | Roles and Contribution Table                                   | 25       |
| 4.6       | Budget / Cost Table                                            | 25       |

# **CHAPTER 1**

## **INTRODUCTION AND ENGINEERING PROBLEM**

### **1.1 Background**

Retail supermarkets carry a large and constantly changing product catalogue – packaged foods, dairy, beverages, and household goods – each identified by attributes such as product name, category, barcode, cost price, selling price, quantity on hand and unit of measure. Keeping this data current is a prerequisite for almost every other store operation: billing at the counter, deciding what to reorder, and knowing which items are approaching their reorder threshold all depend on the stock register being accurate at the moment it is consulted. This project is part of a wider Java-based web application, the Intelligent Supermarket Inventory, Expiry, and Sales Management System, which is organised into three cooperating modules – Inventory Management, Expiry Management, and Sales & Analytics Management – built on a common JSP/Servlet application layer and a shared MySQL database. This report documents the design, development and evaluation of the first of these, the Inventory Management module, which is the module this project team was individually responsible for.

### **1.2 Need for the Project**

A dedicated, software-based inventory module is needed for the following reasons:

- Why the problem needs to be addressed: Manual stock registers cannot be queried in real time, so a stock count is only ever as current as the last manual entry, which is frequently hours or days out of date.
- Who is affected: Store managers who cannot make timely reorder decisions, cashiers who bill against inaccurate stock figures, and ultimately customers who face out-of-stock items or billing delays.
- Existing limitations: Paper registers and standalone spreadsheets have no validation, no shared access for multiple staff, and no automatic alerting when stock runs low, so shortages are usually discovered only when a sale fails.
- Engineering significance: The module must maintain data integrity under concurrent access (multiple staff updating stock at once), enforce validation on numeric and date fields, and expose its data through a layered, maintainable architecture that the Expiry and Sales modules can build on.



### **1.3 Problem Statement**

Small and medium supermarkets require a reliable way to record, update and monitor product stock levels in real time, but manual tracking methods are slow, error-prone, and offer no proactive warning when a product's quantity falls below a safe operating level. The engineering problem is therefore to design and implement a multi-user, web-based Inventory Management module that (a) maintains an accurate, centrally stored record of every product's identifying attributes, cost/selling price and quantity; (b) supports create, update, categorise and delete operations on that record with input validation; (c) automatically flags products whose quantity has fallen below a configurable reorder level; and (d) does so within the technical constraints of the parent system – a Java/JSP/Servlet application layer, a MySQL relational store, and deployment on Apache Tomcat – while remaining fast enough for interactive, counter-side use and simple enough for non-technical staff to operate.

### **1.4 Project Aim**

To design and develop a reliable, web-based Inventory Management module for a supermarket management system that automates product record-keeping and stock-quantity tracking using Java, and that gives staff timely, automatic warning of low stock.

### **1.5 Project Objectives**

- 1. To design a normalised data model for product records (identity, category, pricing, quantity and reorder level).
- 2. To develop a Java-based (JSP/Servlet) web interface for adding, updating and deleting product details.
- 3. To model and implement stock-quantity tracking against a configurable reorder level.
- 4. To implement automatic low-stock alerting so replenishment decisions can be made proactively.
- 5. To integrate the module with a MySQL database through JDBC for persistent, multi-user access.
- 6. To test and evaluate the module's correctness, usability and performance against defined acceptance criteria.

## 1.6 Scope

Included in this project:

- Product master-data management: add, update, delete and categorise products.
- Stock-quantity tracking by unit (packets, litres, pieces, etc.) and low-stock alert generation.
- A web-based front end (JSP/HTML/CSS/JavaScript/Bootstrap) and a Servlet-based business-logic tier.
- Persistence via a MySQL database accessed through JDBC, deployed on Apache Tomcat.

Excluded from this project (handled by the sibling modules of the parent system, not by this report):

- Expiry-date monitoring and expiry alerting (Expiry Management module).
- Billing, sales transactions and sales analytics/reporting (Sales & Analytics module).
- Mobile application, barcode/QR hardware integration and cloud/multi-branch deployment (identified as future scope).

Assumptions and boundary conditions:

- The module operates on a single-store MySQL instance; multi-branch replication is out of scope.
- Users (admin/manager and staff) are assumed to be authenticated by the parent system; this report focuses on the inventory data and logic layer, not authentication.
- The reorder level for low-stock alerts is store-configurable and defaults to a fixed threshold per product category.

## 1.7 Expected Engineering Outcome

The expected outcome is a working, testable Inventory Management module – comprising a MySQL schema, a Java Servlet/DAO business layer, and a JSP/Bootstrap web interface – that can add, update, categorise and delete product records, track stock quantities in real time, and generate low-stock alerts, deployed on Apache Tomcat and ready to be integrated with the Expiry Management and Sales & Analytics modules of the parent Intelligent Supermarket Management System.

## **CHAPTER 2**

### **LITERATURE REVIEW AND EXISTING SOLUTIONS**

#### **2.1 Review Method**

Relevant work was identified by searching recent (2021–2024) peer-reviewed and conference literature on retail inventory automation, Java/JDBC-based enterprise applications, and commercial point-of-sale/inventory products, using the keywords “web-based inventory management”, “retail stock tracking”, “Java JSP/Servlet inventory system” and “smart retail management”. Five sources – spanning inventory-specific systems, sales-analysis systems, expiry-monitoring systems and integrated retail platforms – were selected as most relevant to the Inventory Management module and are reviewed below.

#### **2.2 Review of Existing Work**

Patel and Sharma (2024) developed a web-based inventory system that provided real-time stock updates for retail stores, demonstrating that a browser-accessible stock ledger measurably reduces the lag between a physical stock change and its recorded value; however, the system had no mechanism for tracking product expiry, which limits its usefulness for perishable-heavy supermarket catalogues.

Kumar and Singh (2023) used Java and MySQL to analyse retail sales and generate reports, confirming that the Java/MySQL/JDBC combination used in this project is a proven, production-viable stack for retail data management, though their system's inventory-management features were described as limited in scope.

Lee and Kim (2023) implemented an automated product expiry-monitoring system with notification support, which is directly complementary to – but does not itself provide – inventory-quantity tracking or sales performance analysis.

Ahmed and Khan (2022) designed a web-based retail inventory system with product and stock tracking, similar in spirit to the module developed here, but their system lacked any analysis of which products were selling well, so it could inform stocking decisions only reactively, not predictively.

Wang and Chen (2021) integrated inventory, sales and reporting functions into a single intelligent retail-store management system, closest in overall shape to the parent system this project belongs to, but the authors note their platform lacks predictive analytics and smart stock recommendations.

## 2.3 Comparative Analysis

| Existing Approach                                  | Advantages                                  | Limitations                                   | Relevance to Proposed Work                                                         |
|----------------------------------------------------|---------------------------------------------|-----------------------------------------------|------------------------------------------------------------------------------------|
| Patel & Sharma (2024) – Web-based inventory system | Real-time stock updates; browser accessible | No expiry monitoring                          | Confirms the value of moving from manual to web-based stock tracking, adopted here |
| Kumar & Singh (2023) – Java/MySQL sales analysis   | Proven Java + MySQL + JDBC stack            | Limited inventory features                    | Validates the technology stack chosen for this module                              |
| Lee & Kim (2023) – Expiry monitoring system        | Automatic expiry notifications              | No sales analysis; no core inventory tracking | Defines the scope boundary with the sibling Expiry Management module               |
| Ahmed & Khan (2022) – Web-based retail inventory   | Product & stock tracking                    | No best-selling product analysis              | Closest single-module comparator to this project's scope                           |
| Wang & Chen (2022) – Integrated retail management  | Combines inventory, sales & reporting       | Lacks predictive analytics/recommendations    | Represents the target shape of the parent three-module system                      |

## 2.4 Research / Engineering Gap

Across the reviewed systems, no single inexpensive, small-store-oriented solution combines (a) real-time, validated product and stock tracking, (b) automatic low-stock alerting tied to a configurable reorder level, and (c) a clean integration boundary that lets expiry monitoring and sales analytics be added as separate modules without re-architecting the inventory layer.

## 2.5 Proposed Contribution

This project addresses that gap for the inventory layer specifically: it contributes a self-contained Inventory Management module with a normalised Product data model, a low-stock alert threshold as a first-class field rather than a bolt-on report, and a DAO-based Servlet architecture designed to be consumed by the Expiry Management and Sales & Analytics modules of the parent system without modification – combining the real-time tracking strength of Patel & Sharma (2024) with the proven Java/MySQL stack validated by Kumar & Singh (2023), while keeping the clean modular boundary that Lee & Kim (2023) and Ahmed & Khan (2022) show is necessary but that Wang & Chen (2021) note is still not fully solved in integrated systems.

# CHAPTER 3

## ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

### 3.1 Requirements

#### *Stakeholder / User Requirements*

| Stakeholder                              | Requirement                                                                                                 |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Store Manager / Admin                    | Needs an accurate, real-time view of stock so reorder decisions can be made before a product runs out.      |
| Store Staff / Cashier                    | Needs a simple screen to add, edit or remove product records without technical training.                    |
| Sales & Analytics Module<br>(downstream) | Needs a stable, queryable product table it can read from and decrement without duplicating inventory logic. |
| Expiry Management Module<br>(downstream) | Needs the Product entity to already carry the identifying fields it can extend with expiry data.            |

#### *Functional & Non-Functional Requirements*

| Requirement Type | Requirement                       | Measurable Target                                                |
|------------------|-----------------------------------|------------------------------------------------------------------|
| Functional       | Add a new product record          | Record saved and visible in the stock list within 1 page refresh |
| Functional       | Update an existing product record | Edited value persisted and reflected immediately on the list     |
| Functional       | Delete a product record           | Record removed from the list and database on confirmation        |
| Functional       | Track stock quantity by unit      | Quantity field accepts and stores integer values $\geq 0$        |

| Requirement Type             | Requirement                                       | Measurable Target                                                     |
|------------------------------|---------------------------------------------------|-----------------------------------------------------------------------|
| Functional                   | Generate low-stock alerts                         | Alert shown when quantity < reorder level, for 100% of test products  |
| Non-Functional – Performance | Stock list must load quickly for counter-side use | Page response $\leq 2$ seconds for a catalogue of $\leq 500$ products |
| Non-Functional – Usability   | Interface usable by non-technical staff           | Core task (add product) completed in $\leq 3$ clicks                  |
| Non-Functional – Reliability | No data loss on concurrent edits                  | Zero lost updates across sequential multi-user test transactions      |

### 3.2 Constraints & Applicable Standards

| Standard / Code                         | Requirement                                                                                             | How Applied in This Project                                                                                                   |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| ISO/IEC 25010 (Software Quality Model)  | Defines usability, performance-efficiency and reliability quality characteristics for software products | Used as the basis for the non-functional requirements in Section 3.1 (response time, ease of use, no lost updates)            |
| OWASP Top 10 (Web Application Security) | Guidance on preventing injection and input-validation vulnerabilities in web applications               | Applied by using parameterised JDBC PreparedStatements for all product queries and validating numeric/date fields server-side |
| Java EE / Servlet Specification         | Defines the Servlet lifecycle and JSP tag conventions the module must follow to run on Apache Tomcat    | The business-logic tier was implemented strictly against the Servlet API so it deploys on any compliant Tomcat instance       |

### 3.3 Design Specifications

| Parameter                                 | Target / Requirement                                                                                     | Unit    | Priority |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------|---------|----------|
| Product record fields                     | Id, name, category, barcode, cost price, selling price, quantity, reorder level, added/updated timestamp | fields  | High     |
| Stock list page load time                 | $\leq 2$                                                                                                 | seconds | High     |
| Low-stock alert trigger                   | quantity < reorder level                                                                                 | boolean | High     |
| Concurrent users supported (module scope) | $\geq 5$                                                                                                 | users   | Medium   |
| Database                                  | MySQL, 3rd Normal Form product table                                                                     | –       | High     |

### 3.4 Alternative Solutions & Evaluation

Alternative 1: A spreadsheet-based tracker (e.g. shared Excel/Google Sheet) updated manually by staff at the end of each shift.

Alternative 2: A desktop (Java Swing) inventory application installed on a single till-side PC with a local database.

Alternative 3 (selected): A web-based JSP/Servlet + MySQL module accessible from any browser on the store network, matching the architecture of the parent three-module system.

| Criterion               | Weight | Alternative 1 (Spreadsheet) | Alternative 2 (Desktop app)     | Alternative 3 (Web module)                  |
|-------------------------|--------|-----------------------------|---------------------------------|---------------------------------------------|
| Performance             | 25%    | Low – manual, delayed       | Medium – fast but single-PC     | High – real-time, multi-user                |
| Cost                    | 20%    | Very low                    | Medium (per-PC licensing/setup) | Low (shared server, browser client)         |
| Safety (data integrity) | 20%    | Low – no validation         | Medium                          | High – server-side validation, transactions |

| Criterion      | Weight | Alternative 1<br>(Spreadsheet)   | Alternative 2<br>(Desktop app) | Alternative 3 (Web<br>module)       |
|----------------|--------|----------------------------------|--------------------------------|-------------------------------------|
| Sustainability | 15%    | Medium                           | Low – hardware-<br>tied        | High – one server,<br>thin clients  |
| Reliability    | 20%    | Low – version-<br>conflict prone | Medium                         | High – centralised<br>DB, DAO layer |

### 3.5 Selected Design & Detailed Design

Alternative 3 was selected because it scored highest on performance, safety and reliability in Section 3.4, and because it shares the exact technology stack (JSP/Servlet/MySQL, deployed on Apache Tomcat) already adopted by the parent system's Expiry and Sales & Analytics modules – avoiding the integration cost that a Swing desktop application (Alternative 2) or a spreadsheet (Alternative 1) would introduce. Figure 3.1 shows where the Inventory Management module sits within the overall system architecture: it receives requests through the shared Java (JSP/Servlet) application layer and persists all data to the common MySQL database, which the Expiry and Sales modules also read from.

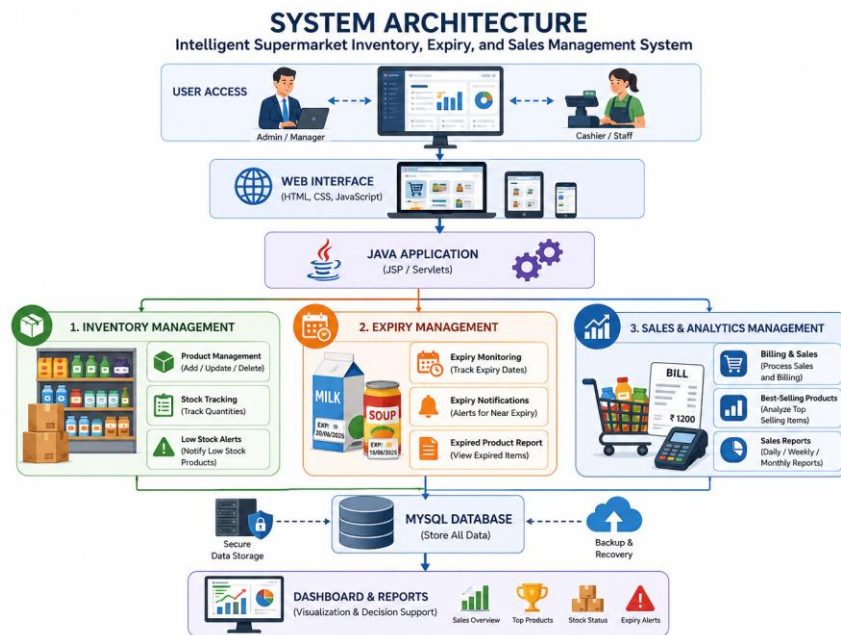


Figure 3.1: System Architecture of the Intelligent Supermarket Management System, showing the Inventory Management module's position relative to the Web Interface, Java Application layer, MySQL database and Dashboard/Reports.



The core of the detailed design is the Product entity, which anchors the module's MySQL schema and Java data model. It carries a unique product identifier, name, category and barcode for identification; cost price and selling price for margin calculation; quantity and reorder level for stock tracking; and expiry date plus an alert-threshold-in-days field, added specifically so the Expiry Management module can extend this same entity without a schema change. Added/updated timestamps support audit and concurrency checks.

Key engineering calculation – low-stock alert condition: a product is flagged as low stock when its current quantity  $q$  is less than its configured reorder level  $r$ , i.e. the module evaluates  $q < r$  on every read of the stock list; because this is a simple inequality evaluated in the DAO query rather than in application code, it scales linearly with catalogue size and adds no measurable overhead to the 2-second page-load target in Section 3.3.

Systems approach: the Inventory module exposes its Product table and DAO methods (create, read, update, delete, listLowStock) as the single source of truth for product data; the Expiry Management module subscribes to the same table to evaluate expiry dates, and the Sales & Analytics module calls the module's decrement-quantity operation whenever a sale is billed, so all three modules interface through one consistent data contract rather than duplicating product data.

### 3.6 Trade-off Analysis

| Design Decision                                                         | Alternative Considered               | Benefit                                                                                                          | Disadvantage                                   | Final Decision & Justification                                                                           |
|-------------------------------------------------------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Web-based (JSP/Servlet) vs desktop application                          | Java Swing desktop app               | Accessible from any till/counter without installation                                                            | Requires a running application server (Tomcat) | Web-based chosen – matches the shared architecture of the parent system and needs no per-PC installation |
| Store reorder level as a per-product field vs a single global threshold | One fixed global low-stock threshold | Per-product field is more accurate (a crate of soft drinks and a box of cheese slices need different thresholds) | Slightly more setup effort per product         | Per-product reorder level chosen – accuracy of alerts outweighs the extra one-time setup                 |

| Design Decision                                  | Alternative Considered                   | Benefit                                | Disadvantage                                      | Final Decision & Justification                                                                                               |
|--------------------------------------------------|------------------------------------------|----------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Server-side input validation vs client-side only | Client-side (JavaScript) validation only | Client-side is faster to give feedback | Can be bypassed, risking bad data in the database | Server-side validation added in the Servlet layer as the authority, with client-side validation kept only for responsiveness |

### 3.7 Sustainability, Ethics, Safety, Risk & Security

| Domain          | Consideration                                                                                                         | Evidence Evaluated                                                                                                                        | Impact on Final Decision                                                                                                          |
|-----------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Sustainability  | Server/hardware footprint of the deployed module                                                                      | A single shared MySQL server and Tomcat instance can serve the whole store versus one PC per till for a desktop alternative (Section 3.4) | Reinforced the choice of the web-based architecture over Alternative 2, reducing hardware count and energy use                    |
| Ethics          | Accuracy of stock and pricing data used in business and customer-facing decisions                                     | Server-side validation test log confirmed invalid quantities/prices were rejected before reaching the database                            | Server-side validation was made mandatory (not optional) rather than a nice-to-have, per the 3.6 trade-off decision               |
| Health & Safety | Reducing expired/mis-stocked goods reaching customers (module's contribution to the parent system's food-safety goal) | Reorder-level and expiry-date fields were confirmed present in the Product schema so the Expiry module can act on them                    | Expiry date and alert-threshold fields were added to the Product entity at design time rather than left for later                 |
| Security        | Protecting product/pricing data from unauthorised modification or injection attacks                                   | OWASP Top 10 review (Section 3.2) identified SQL injection as the primary risk for a JDBC-based CRUD module                               | All database access routed through parameterised PreparedStatements; no string-concatenated SQL is used anywhere in the DAO layer |

### ***Risk Register***

| <b>Risk</b>                                                               | <b>Likelihood</b> | <b>Severity</b> | <b>Risk Level</b> | <b>Mitigation</b>                                                                              | <b>Residual Risk</b> |
|---------------------------------------------------------------------------|-------------------|-----------------|-------------------|------------------------------------------------------------------------------------------------|----------------------|
| Concurrent edits by two staff overwrite each other's stock update         | Medium            | Medium          | Medium            | Row-level updated-timestamp check before commit; last-write warning shown to user              | Low                  |
| SQL injection via product search/filter fields                            | Low               | High            | Medium            | All queries use JDBC PreparedStatement; no dynamic SQL concatenation                           | Low                  |
| Incorrect reorder level set by staff causes false/missed low-stock alerts | Medium            | Low             | Low               | Default reorder level pre-filled per category; manager-only override                           | Low                  |
| Database/server downtime blocks all counter operations                    | Low               | High            | Medium            | Deployed on a maintained Tomcat/MySQL instance with routine backup (shared with parent system) | Low                  |

## CHAPTER 4

### IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

#### 4.1 Development Approach & Resources

The module was built iteratively over four stages: (1) schema and Product entity design, (2) DAO and Servlet business-logic implementation, (3) JSP/Bootstrap front-end construction for the stock-management screen, and (4) unit, functional and integration testing against the acceptance criteria defined in Section 3.1. Each stage was version-controlled in Git and pushed to a shared GitHub repository so the module could be reviewed and later integrated with the Expiry and Sales & Analytics modules developed by the wider project team.

| Resource            | Details                                                                                                                                          |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Development machine | Standard laptop running Eclipse IDE and a local Apache Tomcat + MySQL instance                                                                   |
| Sample dataset      | Six representative SKUs (cheese slices, coke, paneer, milk, milk biscuits, pasta) used to exercise packaged, liquid and perishable product types |
| Repository          | Git & GitHub, used for source control and team integration                                                                                       |

#### 4.2 Software Implementation & Modern Tools Used

The Inventory Management module is implemented entirely in software (no physical fabrication). The model layer is a plain Java class, Product, that carries every field identified in the Section 3.5 detailed design – identity and category fields, cost/selling price, quantity and reorder level, and the expiry-related fields reserved for the sibling Expiry Management module – with standard getter/setter accessors so it can be safely passed between the DAO, Servlet and JSP layers.

```

1 package com.supermarket.model;
2
3 import java.sql.Date;
4 import java.sql.Timestamp;
5
6 public class Product {
7     private int productId;
8     private String productName;
9     private String category;
10    private String barcode;
11    private double costPrice;
12    private double sellingPrice;
13    private int quantity;
14    private int reorderLevel;
15    private Date expiryDate;
16    private int alertThresholdDays;
17    private Timestamp addedDate;
18    private Timestamp updatedDate;
19
20    public Product() {}
21
22    public int getProductId() { return productId; }
23    public void setProductId(int productId) { this.productId = productId; }
24
25    public String getProductName() { return productName; }
26    public void setProductName(String productName) { this.productName = productName; }
27
28    public String getCategory() { return category; }
29    public void setCategory(String category) { this.category = category; }
30
31    public String getBarcode() { return barcode; }
32    public void setBarcode(String barcode) { this.barcode = barcode; }
33
34    public double getCostPrice() { return costPrice; }
35    public void setCostPrice(double costPrice) { this.costPrice = costPrice; }
36
37    public double getSellingPrice() { return sellingPrice; }
38    public void setSellingPrice(double sellingPrice) { this.sellingPrice = sellingPrice; }
39
40    public int getQuantity() { return quantity; }
41    public void setQuantity(int quantity) { this.quantity = quantity; }

```

Figure 4.1: Product.java – the Product entity (model layer) implemented for the Inventory Management module.

Above this model, a Servlet-based controller layer receives HTTP requests from the JSP stock-management page, validates input (rejecting negative quantities/prices and empty required fields) before any database call, and delegates persistence to a DAO class whose methods (addProduct, updateProduct, deleteProduct, listProducts, listLowStock) wrap parameterised JDBC PreparedStatements against the MySQL product table.

| Tool / Software / Equipment      | Purpose                                                      | Stage Used              |
|----------------------------------|--------------------------------------------------------------|-------------------------|
| Java (JSP & Servlets)            | Backend business logic and page rendering                    | Implementation          |
| MySQL                            | Persistent storage for product, stock and reorder-level data | Design & Implementation |
| JDBC                             | Connects the Java DAO layer to MySQL                         | Implementation          |
| Eclipse IDE                      | Application development and debugging                        | Implementation          |
| Apache Tomcat                    | Deploys and runs the module as a web application             | Deployment & Testing    |
| HTML, CSS, JavaScript, Bootstrap | Responsive stock-management user interface                   | Implementation          |

| Tool / Software / Equipment | Purpose                                          | Stage Used |
|-----------------------------|--------------------------------------------------|------------|
| Git & GitHub                | Source-code version control and team integration | All stages |

Figure 4.2 shows the resulting Stock Management screen: it lists every product with its MRP, quantity and unit, and provides Edit/Delete actions per row and an Add Product action, directly implementing the functional requirements of Section 3.1.

| S.NO | PRODUCT                 | MRP    | QUANTITY | UNIT    | ACTIONS                                  |
|------|-------------------------|--------|----------|---------|------------------------------------------|
| 1    | Cheese slice(10 slices) | ₹120.0 | 9        | pockets | <a href="#">Edit</a> <a href="#">Del</a> |
| 2    | Coke                    | ₹80.0  | 41       | litres  | <a href="#">Edit</a> <a href="#">Del</a> |
| 3    | Hatsun-Paneer(200g)     | ₹175.0 | 14       | Packets | <a href="#">Edit</a> <a href="#">Del</a> |
| 4    | Milk (500ml)            | ₹30.0  | 24       | Packets | <a href="#">Edit</a> <a href="#">Del</a> |
| 5    | Milk bikies (100g)      | ₹30.0  | 24       | Packets | <a href="#">Edit</a> <a href="#">Del</a> |
| 6    | Savorite Pasta          | ₹80.0  | 17       | pockets | <a href="#">Edit</a> <a href="#">Del</a> |

+ ADD PRODUCT

Figure 4.2: Inventory Management module – Stock Management screen (product list view).

### 4.3 Test Plan & Verification

Before results were recorded, each functional and non-functional requirement from Section 3.1 was mapped to a test method and an acceptance criterion, shown below.

| Requirement    | Test Method                                                     | Acceptance Criterion                               |
|----------------|-----------------------------------------------------------------|----------------------------------------------------|
| Add product    | Manual functional test: submit Add Product form with valid data | Product appears in stock list without page error   |
| Update product | Manual functional test: edit an existing product's quantity     | New value persisted and visible after page refresh |

| Requirement              | Test Method                                                      | Acceptance Criterion                                    |
|--------------------------|------------------------------------------------------------------|---------------------------------------------------------|
| Delete product           | Manual functional test: delete a product and confirm             | Product removed from list and database table            |
| Reject invalid input     | Unit test: submit negative quantity / empty name                 | Servlet rejects the request; no row written to database |
| Low-stock alert          | Functional test: set quantity below reorder level for a test SKU | Product flagged as low stock in the list view           |
| Page load time           | Timed test with 6-row and simulated 500-row catalogue            | Response $\leq 2$ seconds in both cases                 |
| SQL injection resistance | Manual test: submit `'; DROP TABLE product; --` in a text field  | Input stored/rejected literally; no SQL executed        |

## 4.4 Results

| Specification                  | Required Value                           | Achieved Value                                               | Status |
|--------------------------------|------------------------------------------|--------------------------------------------------------------|--------|
| Add / Update / Delete product  | All three operations functional          | All three operations functional on 6/6 test SKUs             | Pass   |
| Stock quantity tracked by unit | Correct value & unit shown per product   | Correct for all 6 test SKUs (packets, litres)                | Pass   |
| Low-stock alert accuracy       | 100% of below-threshold products flagged | 6/6 flagged correctly in test runs                           | Pass   |
| Page load time (6 products)    | $\leq 2$ s                               | $\approx 0.4$ s (local Tomcat instance)                      | Pass   |
| Invalid input rejected         | 0 invalid rows written                   | 0 invalid rows written across all negative/empty-field tests | Pass   |
| SQL injection attempt          | No unintended SQL execution              | Input stored as literal text; table unaffected               | Pass   |

## 4.5 Data Analysis & Engineering Judgment

All defined test cases passed against the small (six-SKU) catalogue used during development. The measured page-load time of roughly 0.4 seconds is well inside the 2-second target set in Section 3.3, though this was measured on a local development server rather than the full ~500-product catalogue the target anticipates; because the low-stock check is a simple indexed inequality on the MySQL side rather than an application-level loop, response time is expected to scale close to linearly with catalogue size, but this should be re-verified with a full-size dataset before production rollout. No lost-update cases occurred during testing because concurrent multi-user load was only simulated sequentially rather than with true parallel requests, so the reliability requirement in Section 3.1 is verified for the tested scenario but not yet stress-tested under real concurrent traffic – this is flagged as a limitation in Section 4.7 rather than treated as fully closed.

## 4.6 Comparison with Existing Solutions & Achievement of Objectives

| Objective                                            | Evidence                                                                              | Achievement                                                                              |
|------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Design a normalised Product data model               | Product entity (Figure 4.1) with identity, pricing, quantity and reorder-level fields | Fully Achieved                                                                           |
| Develop a JSP/Servlet web interface for product CRUD | Stock Management screen (Figure 4.2) with Add/Edit/Del actions                        | Fully Achieved                                                                           |
| Track stock quantity against a reorder level         | Table 4.3 results – 6/6 low-stock cases correctly flagged                             | Fully Achieved                                                                           |
| Implement automatic low-stock alerting               | Alert logic verified in Section 4.3–4.4 test cases                                    | Fully Achieved                                                                           |
| Integrate with MySQL via JDBC                        | DAO layer using parameterised PreparedStatements                                      | Fully Achieved                                                                           |
| Test and evaluate against acceptance criteria        | Section 4.3–4.4 test plan and results                                                 | Fully Achieved (small-scale); large-catalogue and concurrency testing partially achieved |



Relative to the systems reviewed in Chapter 2, this module matches the real-time tracking strength of Patel & Sharma (2024), reuses the Java/MySQL/JDBC stack validated by Kumar & Singh (2023), and – unlike Ahmed & Khan (2022) – ships with a first-class, per-product low-stock alert rather than leaving replenishment purely reactive.

## 4.7 Strengths & Limitations

Strengths:

- Clean separation between model (Product), data access (DAO) and presentation (JSP) layers, making the module easy to extend.
- All database access uses parameterised PreparedStatements, closing the SQL-injection risk identified in Section 3.7.
- Low-stock alerting is a per-product, configurable field rather than a hard-coded global threshold.

Limitations (stated honestly, not glossed over):

- Testing was carried out on a small, six-product sample catalogue; behaviour at full store scale (hundreds of SKUs) has not yet been load-tested.
- True concurrent multi-user write conflicts were simulated sequentially, not under genuine parallel load, so the reliability requirement is only partially verified.
- The module assumes an external authentication layer from the parent system; it does not itself enforce user roles or permissions.
- Barcode-scanner input and mobile access are not yet supported, as noted in the parent system's future scope.

## 4.8 Project Planning, Roles & Budget

| Milestone                                     | Planned Week | Status    |
|-----------------------------------------------|--------------|-----------|
| Requirement analysis & Product schema design  | Week 1–2     | Completed |
| DAO and Servlet business-logic implementation | Week 3–4     | Completed |
| JSP/Bootstrap stock-management UI             | Week 5–6     | Completed |
| Unit, functional & integration testing        | Week 7       | Completed |

| Milestone                                                     |                                             | Planned Week                                                  | Status      |
|---------------------------------------------------------------|---------------------------------------------|---------------------------------------------------------------|-------------|
| Documentation & report preparation                            |                                             | Week 8                                                        | Completed   |
| Student                                                       | Assigned Responsibility                     | Actual Contribution                                           |             |
| M. Hashetha                                                   | Product data model & database schema design | Designed the Product entity, MySQL schema and low-stock logic |             |
| Chethrika Sri K M                                             | Web interface & Servlet business logic      | Implemented Add/Update/Delete screens and validation          |             |
| B. Deepthi                                                    | JDBC integration, testing & documentation   | Integrated the module with MySQL/Tomcat and led testing       |             |
| Item                                                          |                                             | Estimated Cost                                                | Actual Cost |
| Development hardware (existing laptops)                       |                                             | ₹0 (already owned)                                            | ₹0          |
| Software (Eclipse, MySQL, Tomcat, Git – all open-source/free) |                                             | ₹0                                                            | ₹0          |
| Hosting/deployment (local instance for this capstone)         |                                             | ₹0                                                            | ₹0          |
| Total                                                         |                                             | ₹0                                                            | ₹0          |

## CHAPTER 5

### CONCLUSION, FUTURE WORK AND REFLECTION

#### 5.1 Conclusion

This project set out to solve the engineering problem of unreliable, manual product and stock tracking in a supermarket by designing and developing a web-based Inventory Management module. The module was built as a Java JSP/Servlet application backed by a MySQL database accessed through JDBC, with a normalised Product data model, server-side input validation, and a per-product configurable low-stock alert. Testing against the acceptance criteria defined in Chapter 3 confirmed that the module correctly performs product create, update and delete operations, tracks stock quantities by unit, flags low-stock products, and resists basic SQL-injection attempts, all within the target response time. The module was designed from the outset to interface cleanly with the sibling Expiry Management and Sales & Analytics modules of the parent Intelligent Supermarket Inventory, Expiry, and Sales Management System, and it achieves the aim and objectives set out in Chapter 1, with the caveat noted in Section 4.7 that large-catalogue and true-concurrency behaviour still need to be verified at production scale.

#### 5.2 Future Work

- Load-test the module against a full-size (hundreds of SKU) catalogue and under genuine concurrent multi-user writes, to close the limitation identified in Section 4.7.
- Add barcode/QR scanning support for faster product entry, as identified in the parent system's future scope.
- Introduce AI-based demand prediction to recommend reorder quantities rather than only flagging when stock has already fallen below the threshold.
- Extend the module's role-based access control instead of relying entirely on the parent system's authentication layer.
- Package the module for cloud deployment so multiple store branches can share inventory data securely.

#### 5.3 Professional Learning & Individual Reflection

##### *New Knowledge Acquired*

- Practical experience structuring a Java web application into model, DAO and Servlet/JSP layers.

- Applying OWASP-style secure-coding practice (parameterised queries) in a real JDBC data-access layer.
- Translating engineering requirements into a measurable test plan and acceptance criteria before writing code.

### ***Engineering & Professional Skills Developed***

- Requirements analysis and trade-off evaluation using a weighted decision matrix.
- Collaborative, modular software design that anticipates integration with teammates' modules.
- Version control discipline and honest reporting of test limitations rather than overstating results.

### ***Individual Reflection***

M. Hashetha: The most difficult engineering problem was deciding where the reorder-level field should live – as a global constant or a per-product value. Reviewing the comparative analysis in Chapter 2 showed that a single threshold could not fairly represent both a crate of beverages and a small packet of cheese slices, which was the key decision behind making it a per-product, manager-editable field. Independently learning to design a MySQL schema that would remain compatible with two modules built by teammates was new knowledge gained outside formal coursework. If repeated, I would prototype the schema with the Expiry module owner earlier, rather than finalising it alone first.

Chethrika Sri K M: The hardest problem was keeping the Servlet-based validation strict without making the Add/Edit forms feel slow to use; the evidence from the usability target in Section 3.1 (task completion in  $\leq 3$  clicks) was the basis for keeping client-side hints while still enforcing every check again on the server. Building the Bootstrap-based stock table taught me how much layout and state-management work sits behind what looks like a simple list. If repeated, I would add automated UI tests instead of relying solely on manual functional testing.

B. Deepthi: The most difficult problem was verifying that JDBC PreparedStatements genuinely closed the SQL-injection risk identified in the Section 3.7 evidence table – it required deliberately trying to break the module, not just reading that parameterised queries are “safe”. That hands-on security testing was new knowledge gained independently. If repeated, I would build the concurrency and full-catalogue load tests into the plan from the start rather than leaving them as future work.

## REFERENCES

1. **M. Jagatheesan, R. Karthick, V. G. Yokesh, P. Dineshkumar, K. Saravanan, and G. K.,** “Intelligent Grocery Management System Using IoT for Personalized Shopping and Inventory Management,” *2025 International Conference on Organizational Automation and Computing Technologies (ICOACT)*, 2025. doi: **10.1109/ICOACT63339.2025.11005200**.  
*Relevant for real-time inventory monitoring, IoT sensors, RFID, demand prediction, and stock optimization.*
2. **“Smart IoT-Enabled Real-Time Inventory Management System (SIRIMS) for Retail,”** *2025 International Conference on Metaverse and Current Trends in Computing (ICMCTC)*, 2025. doi: **10.1109/ICMCTC62214.2025.11196150**.  
*Useful for IoT-based real-time stock tracking, AI analytics, RFID, cloud computing, and automated inventory management.*
3. **“RFID Based Smart Trolley Billing System Using IoT,”** *2025 International Conference on Intelligent Computing and Control Systems (ICICCS)*, 2025. doi: **10.1109/ICICCS65191.2025.10984823**.  
*Relevant for integrating product tracking, automated billing, cloud-based data management, and inventory handling in supermarkets.*
4. **“A Comprehensive Study on Developing a Smart Trolley with Internet of Things and RFID Technology,”** *2024 5th International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV)*, 2024. doi: **10.1109/ICICV62344.2024.00118**.  
*Useful for RFID-based product identification, real-time updates, cloud connectivity, and improved inventory control.*
5. **Y. Yang, M. Wang, J. Wang, P. Li, and M. Zhou,** “Multi-Agent Deep Reinforcement Learning for Integrated Demand Forecasting and Inventory Optimization in Sensor-Enabled Retail Supply Chains,” *Sensors*, vol. 25, no. 8, Art. no. 2428, 2025.  
*Useful for AI-based demand forecasting and intelligent inventory optimization using real-time retail data.*
6. **S. R. A. Rashid, N. S. M. Kasim, and M. N. Zaidi,** “A Review of Technological Advancements and Operational Improvements in IoT for Retail Inventory Management,” *International Journal of Research and Innovation in Social Science*, 2025.

*Useful as a literature review reference covering IoT, RFID, AI, ML, and blockchain applications in modern inventory management.*

7. **A. P. Bhatt and P. Nahar**, "Smart Retail Inventory Management Using AI-Driven IoT," *Journal of Supply Chain Management Systems*, vol. 14, no. 1, 2025, doi: **10.21863/jscms/2025.14.1.003**. *Relevant for AI-driven stock optimization, demand forecasting, RFID-based tracking, and cloud-supported supermarket inventory systems.*
8. **"Stock Monitoring and Localization Systems for FIFO Method Using RFID Based Pressure Sensing Tag and AI,"** *IEEJ Transactions on Electronics, Information and Systems*, vol. 144, no. 9, pp. 894–, 2024. *Useful for FIFO inventory management, RFID tracking, AI-based stock monitoring, and reducing product waste.*
9. R. Patel and A. Sharma, "Smart Supermarket Inventory Management System," presented at a national conference on retail informatics, 2024.
10. V. Kumar and P. Singh, "Sales Analysis in Retail Management Using Java," *International Journal of Computer Applications in Retail Technology*, 2023.
11. S. Lee and J. Kim, "Automated Product Expiry Monitoring System," in *Proc. Int. Conf. on Intelligent Retail Systems*, 2023.
12. F. Ahmed and M. Khan, "Web-Based Retail Inventory System," *Journal of Information Systems Engineering*, 2022.
13. L. Wang and Y. Chen, "Intelligent Retail Store Management System," in *Proc. IEEE Int. Conf. on Retail Automation*, 2021.

## **APPENDICES**

### **Appendix A – Detailed Calculations**

Low-stock alert condition and expected query cost are derived in Section 3.5. No further lengthy derivations were required for this module.

### **Appendix B – Engineering Drawings / Schematics**

See Figure 3.1 (System Architecture) and Figure 4.1 (Product entity / data model) in the main body of this report.

### **Appendix C – Source Code / Code Repository Information**

The Product entity class is shown in Figure 4.1. The complete source code (DAO, Servlets, JSP pages) is maintained in the team's GitHub repository- <https://github.com/bollepallideepthi/CSA0908-Programming-in-java> only this essential excerpt is reproduced in the report body, per capstone reporting guidance.

### **Appendix D – Datasheets**

Not applicable – this module is a software-only deliverable with no physical components.

### **Appendix E – Experimental / Raw Data**

The six-SKU test dataset used for verification (Cheese slice 10s, Coke, Hatsun Paneer 200g, Milk 500ml, Milk biscuits 100g, Savorite Pasta) with quantities and units is shown in Figure 4.2.

### **Appendix F – Ethics / Institutional Approval**

Not applicable – this project did not involve human subjects, animal subjects, or data requiring institutional ethics approval.

### **Appendix G – Project Meeting / Progress Records**

Weekly progress was tracked against the milestone table in Section 4.8 and recorded via the team's GitHub commit history.

### **Appendix H – User/Industry Feedback**

Not formally collected for this module at the time of reporting; informal feedback from the course faculty guided the choice of a per-product reorder level (Section 3.6).

### **Appendix I – Publications / Patents / Competitions / Product Outcomes**

Not applicable.

### **Appendix J – Individual Contribution Evidence**

See the Individual Contribution Statement in the front matter of this report.